

LIBERTAS

Curso: Sistemas de Informação

Disciplina: Desenvolvimento Web I

Professor: MSc. Jonathan Henrique Jeremias Souza

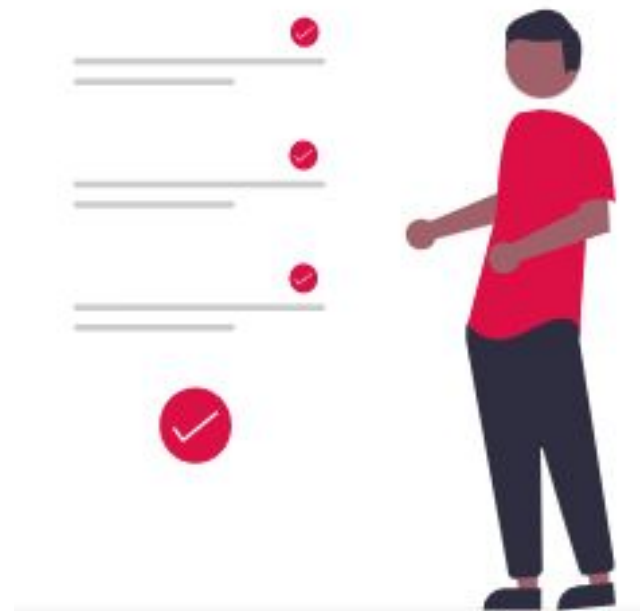
E-mail: jonathansouza@libertas.edu.br



Libertas[®]
FACULDADES INTEGRADAS

AGENDA

- Tipos de Dados em JavaScript
- Interação com o Usuário: alert, prompt e confirm
- Conversão de Tipos e Operadores Matemáticos



TIPOS DE DADOS EM JAVASCRIPT

Todo valor em JavaScript possui um tipo

Um valor em JavaScript é sempre de um determinado tipo. Por exemplo, uma string ou um número.

Existem oito tipos básicos de dados em JavaScript. Nesta aula veremos todos de forma geral.

Depois capítulos falaremos sobre cada um deles em detalhes.

TIPOS DE DADOS EM JAVASCRIPT

Tipagem dinâmica

Podemos colocar qualquer tipo em uma variável. Uma variável pode, em determinado momento, ser uma string e depois armazenar um número.

Linguagens de programação que permitem esse comportamento são chamadas de "dinamicamente tipadas".

Isso significa que existem tipos de dados, mas as variáveis não estão vinculadas a nenhum deles.

```
// sem erro  
let message = "hello";  
message = 123456;
```

TIPOS DE DADOS EM JAVASCRIPT

O tipo Number

O tipo number representa tanto números inteiros quanto números de ponto flutuante.

Há muitas operações para números: multiplicação *, divisão /, adição +, subtração -, e assim por diante.

Além dos números comuns, existem os chamados "valores numéricos especiais": Infinity, -Infinity e NaN.

```
let n = 123;  
n = 12.345;
```

TIPOS DE DADOS EM JAVASCRIPT

Infinity

Infinity representa o infinito matemático ∞ . É um valor especial que é maior que qualquer número.

Podemos obtê-lo como resultado de uma divisão por zero.

Ou simplesmente referenciar diretamente.

```
alert( 1 / 0 ); // Infinity  
  
alert( Infinity ); // Infinity
```

TIPOS DE DADOS EM JAVASCRIPT

NaN

NaN representa um erro computacional. É o resultado de uma operação matemática incorreta ou indefinida.

NaN é "persistente" (sticky). Qualquer operação matemática adicional sobre NaN retorna NaN. Há apenas uma exceção: $\text{NaN} ** 0$ é igual a 1.

```
alert( "not a number" / 2 ); // NaN

alert( NaN + 1 );           // NaN
alert( 3 * NaN );          // NaN
alert( "not a number" / 2 - 1 ); // NaN
```

TIPOS DE DADOS EM JAVASCRIPT

Operações matemáticas são seguras em JavaScript

Fazer cálculos matemáticos em JavaScript é "seguro". Podemos fazer qualquer coisa: dividir por zero, tratar strings não numéricas como números, etc.

O script nunca irá parar com um erro fatal. Na pior das hipóteses, obteremos NaN como resultado.

Valores numéricos especiais pertencem formalmente ao tipo number, mas não são números no sentido comum da palavra.

TIPOS DE DADOS EM JAVASCRIPT

O tipo BigInt

O tipo number não consegue representar com segurança inteiros maiores que $(2^{53} - 1)$, que é 9007199254740991. Fora desse intervalo, haverá erro de precisão.

BigInt foi adicionado à linguagem para representar inteiros de comprimento arbitrário. Um valor BigInt é criado acrescentando n ao final de um inteiro.

```
console.log(9007199254740991 + 1); // 9007199254740992
console.log(9007199254740991 + 2); // 9007199254740992

// o "n" no final significa que é um BigInt
const bigInt = 1234567890123456789012345678901234567890n;
```

TIPOS DE DADOS EM JAVASCRIPT

O tipo String, três tipos de aspas

Uma string em JavaScript deve estar entre aspas. Há três tipos: aspas duplas, aspas simples e backticks.

Aspas duplas e simples são aspas "simples", praticamente não há diferença entre elas em JavaScript. Backticks são aspas de "funcionalidade estendida": permitem incorporar variáveis e expressões com `${...}`.

```
let str    = "Hello";  
let str2   = 'Single quotes are ok too';  
let phrase = `can embed another ${str}`;
```

TIPOS DE DADOS EM JAVASCRIPT

Incorporando expressões com backtick

A expressão dentro de `${...}` é avaliada e o resultado se torna parte da string. Podemos colocar qualquer coisa lá: uma variável ou uma expressão aritmética.

Isso só pode ser feito com backticks. Outras aspas não têm essa funcionalidade. Com aspas duplas ou simples, o `${}` é tratado como texto literal, sem avaliação.

```
let name = "John";

alert( `Hello, ${name}!` );           // Hello, John!
alert( `the result is ${1 + 2}` );   // the result is 3

// aspas duplas não fazem nada com ${}
alert( "the result is ${1 + 2}" );   // the result is ${1 + 2}
```

TIPOS DE DADOS EM JAVASCRIPT

Não existe tipo character em JavaScript

Em algumas linguagens existe um tipo especial para um único caractere. Em C e em Java, ele é chamado de char.

Em JavaScript não existe esse tipo. Há apenas um tipo: string.

Uma string pode consistir de zero caracteres (ser vazia), um caractere ou muitos deles.

TIPOS DE DADOS EM JAVASCRIPT

O tipo Boolean

O tipo boolean tem apenas dois valores: true e false.

É usado para armazenar valores sim/não: true significa "sim, correto" e false significa "não, incorreto".

Valores booleanos também surgem como resultado de comparações.

```
let nameFieldChecked = true; // sim, o campo está marcado
let ageFieldChecked  = false; // não, o campo não está marcado

let isGreater = 4 > 1;
alert( isGreater ); // true
```

TIPOS DE DADOS EM JAVASCRIPT

O valor null

O valor especial null não pertence a nenhum dos tipos descritos. Ele forma um tipo próprio que contém apenas o valor null.

Em JavaScript, null não é uma "referência a um objeto inexistente" ou um "ponteiro nulo" como em outras linguagens.

É apenas um valor especial que representa "nada", "vazio" ou "valor desconhecido".

```
let age = null;  
// age é desconhecida
```

TIPOS DE DADOS EM JAVASCRIPT

O valor undefined

O valor especial undefined também se destaca. Ele forma um tipo próprio, assim como null.

O significado de undefined é "valor não atribuído".

Se uma variável é declarada mas não atribuída, seu valor é undefined.

```
let age;  
alert(age); // undefined
```

TIPOS DE DADOS EM JAVASCRIPT

Atribuir undefined explicitamente não é recomendado

Tecnicamente é possível atribuir undefined a uma variável explicitamente. Mas normalmente não se recomenda fazer isso.

Usa-se null para atribuir um valor "vazio" ou "desconhecido", enquanto undefined é reservado como valor inicial padrão para coisas não atribuídas.

```
let age = 100;  
  
// muda o valor para undefined  
age = undefined;  
  
alert(age); // "undefined"
```


TIPOS DE DADOS EM JAVASCRIPT

Objetos e Symbols

O tipo object é especial. Todos os outros tipos são chamados de "primitivos" porque seus valores contêm apenas uma coisa. Objetos armazenam coleções de dados e entidades mais complexas.

O tipo symbol é usado para criar identificadores únicos para objetos.

Ambos merecem tratamento especial e serão estudados após aprendermos mais sobre os primitivos.

TIPOS DE DADOS EM JAVASCRIPT

O operador typeof

O operador `typeof` retorna o tipo do operando. É útil quando queremos processar valores de tipos diferentes ou fazer uma verificação rápida.

Uma chamada a `typeof x` retorna uma string com o nome do tipo.

Pode ser usado como `typeof x` ou como `typeof(x)` — é a mesma coisa. `typeof` é um operador, não uma função.

TIPOS DE DADOS EM JAVASCRIPT

O operador typeof

```
typeof undefined    // "undefined"  
typeof 0             // "number"  
typeof 10n           // "bigint"  
typeof true         // "boolean"  
typeof "foo"        // "string"  
typeof Symbol("id") // "symbol"  
typeof Math         // "object"  
typeof null         // "object"  
typeof alert        // "function"
```

TIPOS DE DADOS EM JAVASCRIPT

Três casos especiais do typeof

typeof null retorna "object". Isso é um erro oficialmente reconhecido, vindo dos primeiros dias do JavaScript e mantido por compatibilidade. null definitivamente não é um objeto.

typeof alert retorna "function". Funções pertencem ao tipo object, mas typeof as trata de forma diferente, também herança dos primeiros dias da linguagem.

typeof Math retorna "object". Math é um objeto embutido que fornece operações matemáticas.

INTERAÇÃO COM O USUÁRIO: ALERT, PROMPT E CONFIRM

Funções de interação com o usuário no navegador

Como usaremos o navegador como ambiente de demonstração, veremos algumas funções para interagir com o usuário: alert, prompt e confirm.

Essas são funções específicas do navegador para interagir com visitantes.

Todas são modais: pausam a execução do script e não permitem interação com o restante da página até que a janela seja fechada.

INTERAÇÃO COM O USUÁRIO: ALERT, PROMPT E CONFIRM

alert

Exibe uma mensagem e aguarda o usuário pressionar OK.

A mini-janela com a mensagem é chamada de janela modal.

O visitante não pode interagir com o restante da página nem pressionar outros botões até pressionar OK.

```
alert("Hello");
```

INTERAÇÃO COM O USUÁRIO: ALERT, PROMPT E CONFIRM

prompt, sintaxe e argumentos

A função prompt aceita dois argumentos: title (o texto a mostrar ao visitante) e default (valor inicial opcional para o campo de entrada).

Exibe uma janela modal com uma mensagem de texto, um campo de entrada para o visitante e os botões OK/Cancelar.

Os colchetes em torno de default na sintaxe indicam que o parâmetro é opcional.

```
result = prompt(title, [default]);
```

INTERAÇÃO COM O USUÁRIO: ALERT, PROMPT E CONFIRM

prompt, retorno

O visitante pode digitar algo no campo e pressionar OK, nesse caso obtemos o texto no resultado.

Ou cancelar pressionando Cancelar ou Esc, nesse caso obtemos null como resultado.

A chamada ao prompt retorna o texto do campo ou null se a entrada for cancelada.

```
let age = prompt('How old are you?', 100);  
  
alert(`You are ${age} years old!`); // You are 100 years old!
```


INTERAÇÃO COM O USUÁRIO: ALERT, PROMPT E CONFIRM

confirm

A função confirm exibe uma janela modal com uma pergunta e dois botões: OK e Cancelar.

O resultado é true se OK for pressionado e false caso contrário.

É usada para confirmações de ações.

```
result = confirm(question);  
  
let isBoss = confirm("Are you the boss?");  
alert( isBoss ); // true se OK for pressionado
```

INTERAÇÃO COM O USUÁRIO: ALERT, PROMPT E CONFIRM

Resumo e limitações

Cobrimos 3 funções: alert (exibe mensagem), prompt (pede texto, retorna o texto ou null), confirm (exibe pergunta, retorna true ou false).

Todos esses métodos são modais: pausam a execução do script e não permitem interação com o restante da página.

Duas limitações: a localização e a aparência da janela são determinadas pelo navegador, não podemos modificá-las.

CONVERSÃO DE TIPOS E OPERADORES MATEMÁTICOS

Conversão de tipos, visão geral

Na maior parte do tempo, operadores e funções convertem automaticamente os valores para o tipo correto.

Por exemplo, `alert` converte automaticamente qualquer valor para string para exibi-lo. Operações matemáticas convertem valores para numbers.

Também há casos em que precisamos converter um valor explicitamente para o tipo esperado.

CONVERSÃO DE TIPOS E OPERADORES MATEMÁTICOS

Conversão para String

A conversão para string ocorre quando precisamos da forma textual de um valor. Por exemplo, `alert(value)` faz isso para exibir o valor.

Podemos também chamar `String(value)` para converter explicitamente. A conversão para string é em sua maioria óbvia: `false` vira `"false"`, `null` vira `"null"`, e assim por diante.

```
let value = true;
alert(typeof value); // boolean

value = String(value); // agora value é a string "true"
alert(typeof value);   // string
```

CONVERSÃO DE TIPOS E OPERADORES MATEMÁTICOS

Conversão Numérica, automática e explícita

A conversão numérica em funções matemáticas e expressões ocorre automaticamente. Por exemplo, quando a divisão `/` é aplicada a não-números.

Podemos usar `Number(value)` para converter explicitamente.

A conversão explícita é necessária quando lemos um valor de uma fonte baseada em string, como um formulário, mas esperamos um número.

CONVERSÃO DE TIPOS E OPERADORES MATEMÁTICOS

Conversão Numérica, automática e explícita

```
alert( "6" / "2" ); // 3, strings são convertidas para números
```

```
let str = "123";  
alert(typeof str); // string
```

```
let num = Number(str); // vira o número 123  
alert(typeof num);     // number
```

CONVERSÃO DE TIPOS E OPERADORES MATEMÁTICOS

Conversão Numérica, string inválida

Se a string não é um número válido, o resultado da conversão é NaN. A conversão falhou, o valor não pôde ser interpretado como número.

Isso acontece ao tentar converter uma string de texto arbitrária.

```
let age = Number("an arbitrary string instead of a number");  
  
alert(age); // NaN, conversão falhou
```

CONVERSÃO DE TIPOS E OPERADORES MATEMÁTICOS

Regras de conversão numérica

undefined vira NaN.

null vira 0.

true vira 1 e

false vira 0.

Para strings: espaços do início e fim são removidos. String vazia vira 0. Caso contrário, o número é lido. Um erro dá NaN.

CONVERSÃO DE TIPOS E OPERADORES MATEMÁTICOS

Regras de conversão numérica

Importante: null e undefined se comportam de forma diferente — null vira zero, undefined vira NaN.

```
alert( Number(" 123 ") ); // 123
alert( Number("123z") ); // NaN
alert( Number(true) ); // 1
alert( Number(false) ); // 0
```

MUITO OBRIGADO!

Ficou com alguma
dúvida?



MSc. Jonathan Henrique Jeremias Souza
jonathansouza@libertas.edu.br
www.libertas.edu.br



Libertas[®]
FACULDADES INTEGRADAS

Referências Bibliográficas

- KANTOR, Ilya. Data types. The Modern JavaScript Tutorial. Disponível em: <https://javascript.info/types>. Acesso em: 31 mai. 2026.
- KANTOR, Ilya. Interaction: alert, prompt, confirm. The Modern JavaScript Tutorial. Disponível em: <https://javascript.info/alert-prompt-confirm>. Acesso em: 31 mai. 2026.
- KANTOR, Ilya. Type Conversions. The Modern JavaScript Tutorial. Disponível em: <https://javascript.info/type-conversions>. Acesso em: 31 mai. 2026.
- KANTOR, Ilya. Basic operators, maths. The Modern JavaScript Tutorial. Disponível em: <https://javascript.info/operators>. Acesso em: 31 mai. 2026.